WILLIAMS V.S. TUBMAN UNIVERSITY

East Harper, Tubman Town, Maryland County

P.O.BOX 3570, www.tubmanu.edu.lr

College of Engineering and Technology

Department of Computer Science

Course: Software Engineering

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Technical Reference Document

Version 1.0 | March 2026 | CONFIDENTIAL

Prepared for: Computer Lab Management Team, Department of Computer Science

Course: Software Engineering

Project Lead: Princeton R. Zahnmie

Team Size: 6 Members

Academic Supervisor: Prof. Kwitee D. Gaylah

SDLC Methodology: Agile (Scrum)

TABLE OF CONTENTS

1. Executive Summary
2. Project Context & Stakeholders
3. System Overview & Architecture
4. Engineering Principles & Quality Standards
5. Functional Requirements
6. Key Feature Flows
7. Non-Functional Requirements
8. Deployment & Technical Infrastructure
9. Database Schema Overview
10. Agile Development Plan & Timeline
11. Team Roles & Responsibilities
12. Acceptance Criteria
13. Approval & Sign-Off

1. EXECUTIVE SUMMARY

The Tubman University Computer Lab Inventory System is a standalone desktop application developed as a capstone project for the Software Engineering course at William V.S. Tubman University. This document serves as the Software Requirements Specification (SRS) the authoritative technical reference for designing, developing, testing, and deploying the system.

This document translates the operational requirements of the Computer Lab into precise technical language: what the system must do, how it will be built using engineering principles, the technologies involved, and how it will be deployed.

1.1 The Problem This System Solves

The Computer Science Department's laboratories currently manage equipment using manual logbooks, spreadsheets, or informal tracking. The consequences:

1. **Equipment Loss and Misplacement: No centralized record of who borrowed which device, leading to lost laptops, projectors, and peripherals.**
2. **Poor Maintenance Tracking: No systematic record of repairs, servicing schedules, or warranty status, causing unexpected downtime.**
3. **Inefficient Check-Out Process: Manual sign-out sheets are slow, error-prone, and difficult to audit.**
4. **No Real-Time Availability: Students and faculty cannot check equipment availability before visiting the lab.**
5. **Limited Reporting: Department heads cannot generate reports on utilization, loss rates, or budget justification.**
6. **Offline Constraints: Internet connectivity in the lab is unreliable; cloud-based solutions are not viable.**

1.2 What The System Will Deliver

A full-stack desktop application serving as the digital backbone of the Computer Lab enabling:

1. **Equipment Catalog Management: Track all lab assets with serial numbers, warranty status, and location.**
2. **Check-Out / Check-In Workflow: Digital recording of borrowing with due dates and return tracking.**
3. **Condition & Maintenance Tracking: Log equipment status and schedule maintenance tasks.**
4. **Real-Time Availability Dashboard: Show available, checked out, or under repair equipment at a glance.**
5. **Automated Alerts: Notify staff of overdue returns, low stock, and upcoming warranty expirations.**
6. **Reporting & Analytics: Generate reports on utilization, loss/damage rates, and maintenance costs.**
7. **Offline-First Architecture: 100% functional without internet; data stored locally using SQLite.**
8. **Data Backup & Recovery: Automated local backups with one-click restore.**

NOTE: This system is designed as a standalone desktop application using Electron.js, Nest.js, and SQLite. Development follows the Agile (Scrum) methodology as part of the Software Engineering course curriculum.

2. PROJECT CONTEXT & STAKEHOLDERS

2.1 Academic Context

This project is developed as part of the Software Engineering course curriculum at William V.S. Tubman University. The primary learning objectives are:

1. **Apply Engineering Principles: Demonstrate SOLID principles, design patterns, testing strategies, and software architecture.**
2. **Practice Agile Methodology: Experience iterative development, sprint planning, and retrospective practices.**
3. **Collaborative Development: Learn version control, code review, pull request workflows, and team coordination using GitHub.**
4. **Professional Documentation: Produce industry-standard technical documentation suitable for academic evaluation.**
5. **Quality Assurance: Implement automated testing, code quality checks, and continuous integration practices.**

2.2 Stakeholder Map

Stakeholder	Role	Interest in the System
Project Lead / Student Developer	**Technical Lead & Scrum Master**	**System architecture; code quality; team coordination; learning outcomes; project completion**
Academic Supervisor / Professor	**Course Instructor & Evaluator**	**Learning objectives met; engineering principles applied; professional standards; timely delivery**
Lab Technician (End User Persona)	**Primary Daily User**	**Efficient check-out process; accurate equipment tracking; maintenance scheduling; reduced paperwork**
Faculty Member (End User Persona)	**Secondary User**	**Quick equipment reservation; visibility into availability; simple return process**
Student Borrower (End User Persona)	**Occasional User**	**Easy equipment request; clear due dates; return reminders; transparent accountability**
Department Head / Admin	**Reporting User**	**Utilization reports; loss/damage statistics; budget justification data**

2.3 User Classes and Characteristics

Class 1: Lab Technician (Primary End User)

1. **Technical proficiency: Moderate**
2. **Responsibilities: Equipment check-out/check-in; condition updates; maintenance logging; inventory audits**

3. **Access Level: Full system access except user management**
4. **Frequency of Use: Daily during lab hours**

Class 2: Faculty Member (Secondary End User)

1. **Technical proficiency: Basic to Moderate**
2. **Responsibilities: Reserve equipment for classes; check availability; return borrowed items**
3. **Access Level: Check-out/return functions; view availability; limited reporting**
4. **Frequency of Use: Weekly or as needed**

Class 3: Student Borrower (Occasional End User)

1. **Technical proficiency: Basic**
2. **Responsibilities: Request equipment for projects; return items on time; report damage**
3. **Access Level: View availability; request check-out (requires faculty approval); view personal borrowing history**
4. **Frequency of Use: Occasional (project-based)**

Class 4: Department Administrator (Reporting User)

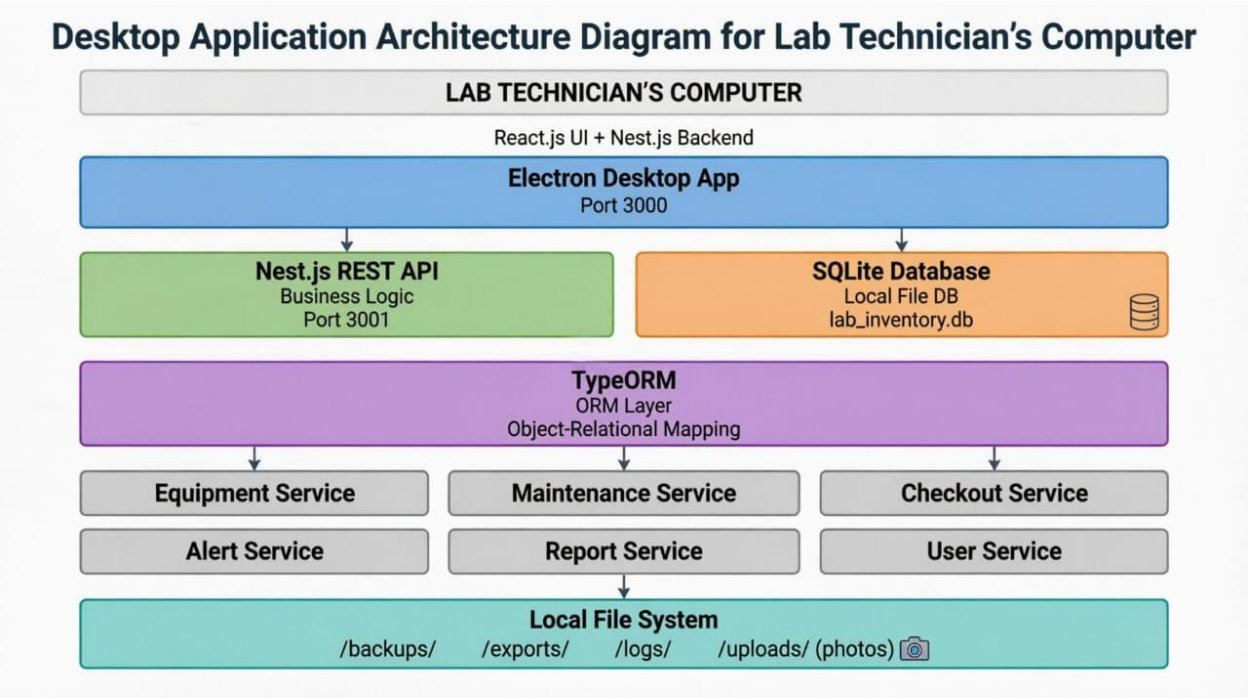
1. **Technical proficiency: Basic**
2. **Responsibilities: Generate utilization reports; review loss/damage statistics; budget planning**
3. **Access Level: Read-only access to reports and dashboards; no modification rights**
4. **Frequency of Use: Monthly or quarterly**

3. SYSTEM OVERVIEW & ARCHITECTURE

3.1 System Description

The Tubman University Computer Lab Inventory System is a multi-tier desktop application with three logical layers: an Electron.js frontend (desktop UI with web technologies), a Nest.js REST API backend (application logic and data management running locally), and a SQLite relational database (persistent local data store). The entire application is packaged as a native desktop executable.

3.2 High-Level Architecture



3.3 Application Architecture Components

Component	Technology	Responsibility
Electron Main Process	**Electron.js + Node.js**	**Desktop window management, native OS integration, auto-updater, file system access**
Electron Renderer	**React.js 18 + TypeScript**	**User interface rendering, user interactions, state management**
Backend API	**Node.js 20 + Nest.js**	**REST API endpoints, business logic, data validation, authentication**
Database	**SQLite 3 + TypeORM**	**Persistent data storage, transactions, indexing**
State Management	**Redux Toolkit / Zustand**	**Frontend state, caching, optimistic updates**
UI Component Library	**Material-UI / Ant Design**	**Pre-built components, themes, responsive layouts**
Backup Service	**Custom Node.js module**	**Automated database backups, compression, rotation**

3.4 Full Technology Stack

Layer	Technology	Justification
Desktop Framework	**Electron.js 28**	**Cross-platform desktop app; single codebase for Windows/Mac/Linux**
Frontend	**React.js 18, TypeScript**	**Component-based UI; type safety; large ecosystem**
Backend API	**Node.js 20 LTS, Nest.js**	**Modular architecture; dependency injection; enterprise-grade patterns**
Authentication	**JWT + bcrypt**	**Secure password hashing; token-based auth; role-based access control**
Database	**SQLite 3**	**Serverless; zero configuration; single file; ACID-compliant; perfect for offline desktop apps**
ORM	**TypeORM 0.3**	**Type-safe queries; automatic migrations; supports SQLite**
Testing	**Jest + React Testing Library**	**Unit testing; integration testing; snapshot testing**
Packaging	**Electron Builder**	**Create installers (.exe, .dmg); code signing; auto-updater**
Version Control	**Git + GitHub**	**Source control; collaboration; issue tracking; CI/CD**

4. ENGINEERING PRINCIPLES & QUALITY STANDARDS

4.1 SOLID Principles Application

Principle	Application in This Project	Benefit
Single Responsibility	**Each Nest.js module handles one domain (Equipment, Checkout, Maintenance); each React component has one purpose**	**Easier testing; clearer code; simpler maintenance**
Open/Closed	**Services can be extended via inheritance/interfaces without modifying existing code**	**Flexible; reduces regression risk**
Liskov Substitution	**Repository interfaces allow swapping SQLite for other databases without changing business logic**	**Testability; future-proofing**
Interface Segregation	**Fine-grained interfaces for services rather than one large interface**	**Cleaner dependencies; easier mocking**
Dependency Inversion	**High-level modules depend on abstractions, not concrete implementations; DI via Nest.js**	**Loose coupling; easier testing**

4.2 DRY (Don't Repeat Yourself)

- **Shared TypeScript interfaces between frontend and backend for API contracts**
- **Reusable React components for forms, tables, modals**
- **Centralized validation logic using class-validator decorators**
- **Common utility functions in a shared /lib directory**

4.3 Security by Design

- **Password hashing with bcrypt (12 salt rounds) at rest**
- **Role-based access control (RBAC) enforced at API middleware level**
- **SQL injection prevention via TypeORM parameterized queries**
- **Input validation and sanitization at all entry points**

4.4 Data Integrity

- **Database transactions for all write operations (ACID compliance)**
- **Foreign key constraints to maintain referential integrity**
- **Audit trail logging for all critical operations**
- **Automated backups with verification and rotation**

4.5 Testability

- **Unit tests for all services and utilities (Jest)**
- **Integration tests for API endpoints (Supertest)**
- **Component tests for critical UI elements (React Testing Library)**
- **Minimum 80% code coverage requirement for merge approval**

5. FUNCTIONAL REQUIREMENTS

Each requirement has a unique ID, a priority (P1 = Critical/Must-Have, P2 = Important/Should-Have, P3 = Nice-to-Have/Could-Have), and the user role it serves.

5.1 User Authentication & Account Management

ID	Requirement	Detailed Description	Priority	Role
FR-AUTH-01	**User Registration**	**System allows Admin to create new user accounts with username, email, password, full name, role, and department**	**P1**	**Admin**
FR-AUTH-02	**User Login**	**Users can log in with username/email and password; system validates credentials and issues session token**	**P1**	**All**
FR-AUTH-03	**Password Security**	**Passwords stored using bcrypt hashing algorithm with minimum 12 salt rounds; never stored in plaintext**	**P1**	**System**

FR-AUTH-04	Role-Based Access Control	Passwords stored using bcrypt hashing algorithm with minimum 12 salt rounds; never stored in plaintext	P1	System
FR-AUTH-05	Session Management	User sessions expire after 8 hours of inactivity; automatic logout with unsaved data warning	P1	System
FR-AUTH-06	User Profile Management	Users can update their own profile information: name, email, phone, password (but not role	P2	All

5.2Equipment Catalog Management

ID	Requirements	Detailed Description	Priority	Roles
FR-EQP-01	Add New Equipment	Lab Technician can add new equipment with serial number, model, purchase date, warranty status, location	P1	Technician
FR-EQP-02	View Equipment Details	Users can view comprehensive details including current status, checkout history, maintenance records	P1	ALL
FR-EQP-03	Edit Equipment Information	Authorized users can modify equipment details except historical transaction data	P1	Technician
FR-EQP-04	Change Equipment Status	Lab Technician can update equipment status: Available, Checked Out, Under Repair, Retired, Lost	P1	Technician
FR-EQP-05	Equipment Search & Filter	Advanced search with filters for category, location, status, and keyword search	P1	ALL
FR-EQP-06	Bulk Equipment Export	Users can export equipment list to CSV or Excel with selected columns and filters	P2	ALL

FR-EQP-07	**Equipment Photo Upload**	**Lab Technician can upload photos of equipment for visual identification**	**P2**	**Technician**

5.3Checkout / Check-In Workflow

ID	Requirement	Detailed Description	Priority	Role
FR-CHK-01	**Equipment Checkout**	**Record equipment borrowing with details: equipment, borrower, checkout date, due date, purpose**	**P1**	**Technician**
FR-CHK-02	**Equipment Return**	**Record equipment return with condition check and optional damage notes**	**P1**	**Technician**
FR-CHK-03	**Overdue Alert**	**System automatically identifies overdue checkouts and displays alerts to Lab Technician**	**P1**	**System**
FR-CHK-04	**Checkout History**	**View complete history of all equipment checkouts with filtering and sorting**	**P1**	**All**
FR-CHK-05	**Borrower Management**	**Lab Technician can view borrowing history per user, including current checkouts and past returns**	**P2**	**Technician**

5.4 Maintenance & Condition Tracking

ID	Requirement	Detailed Description	Priority	Role
FR-MNT-01	**Schedule Maintenance**	**Lab Technician can schedule maintenance tasks with date, type, and technician assignment**	**P1**	**Technician**
FR-MNT-02	**Record Maintenance Completion**	**Log completed maintenance with details: actual date, work performed, cost, parts used**	**P1**	**Technician**

FR-MNT-03	**Maintenance History**	**View complete maintenance history for any equipment item**	**P1**	**Technician**
FR-MNT-04	**Warranty Tracking**	**System tracks warranty expiration dates and alerts of upcoming expirations**	**P2**	**Technician**

5.5Alerts & Notifications

ID	Requirement	Detailed Description	Priority	Role
FR-ALRT-01	**Overdue Checkout Alert**	**System automatically identifies overdue checkouts and displays alerts**	**P1**	**Technician**
FR-ALRT-02	**Low Consumables Alert**	**Alert when consumable items fall below minimum stock level**	**P2**	**Technician**
FR-ALRT-03	**Warranty Expiry Alert**	**Alert for equipment warranties approaching expiration date**	**P2**	**Technician**
FR-ALRT-04	**Alert Configuration**	**Users can customize alert thresholds and notification methods**	**P2**	**Admin**

5.6Reporting & Analytics

ID	Requirement	Detailed Description	Priority	Role
FR-RPT-01	**Equipment Utilization Report**	**Report showing equipment usage frequency, checkout duration, and idle time**	**P1**	**Admin**
FR-RPT-02	**Loss/Damage Report**	**Report on equipment lost, damaged, or retired with cost impact**	**P1**	**Admin**
FR-RPT-03	**Maintenance Cost Report**	**Report on maintenance spending by equipment category or time period**	**P1**	**Admin**
FR-RPT-04	**Export Reports**	**Export reports to CSV and Excel formats**	**P1**	**Admin**

5.7 Data Backup & Recovery

ID	Requirement	Detailed Description	Priority	Role
FR-BKP-01	Automated Backup	System automatically creates database backups on a configurable schedule	P1	System
FR-BKP-02	Manual Backup	Users can trigger immediate backup at any time	P1	Admin
FR-BKP-03	Backup Restoration	Users can restore database from a backup file with confirmation	P1	Admin
FR-BKP-04	Backup Verification	System validates backup file integrity after creation	P2	System

6. KEY FEATURE FLOWS

6.1 New Equipment Addition Flow

STEP 1: Lab Technician navigates to Equipment module.

STEP 2: Technician clicks "Add New Equipment" button.

STEP 3: System displays form: Name, Category, Serial Number, Model, Purchase Date, Warranty, Location, Condition.

STEP 4: Technician fills required fields; client validates serial number uniqueness.

STEP 5: POST /api/equipment with form data.

STEP 6: Backend validates data; checks duplicate Serial Number.

STEP 7: Insert record into equipment table.

STEP 8: Generate audit log: "Equipment added by [User]".

STEP 9: Frontend shows success notification; redirects to equipment list.

6.2 Equipment Checkout Flow

STEP 1: Lab Technician navigates to Checkout module.

STEP 2: Technician clicks "New Checkout" button.

STEP 3: System displays form: Equipment selector, Borrower selector, Due Date, Purpose.

STEP 4: Technician selects equipment; system checks status == 'Available'.

STEP 5: Technician selects borrower (search by name/ID).

STEP 6: Technician enters due date (default +7 days).

STEP 7: POST /api/checkouts with payload.

STEP 8: Backend validates permission; starts database transaction.

STEP 9: Insert record into checkouts table; Update equipment status to 'Checked Out'.

STEP 10: Commit transaction; Generate audit log.

STEP 11: Frontend shows success; prints checkout receipt.

6.3 Equipment Return Flow

STEP 1: Lab Technician navigates to Checkout module > Returns tab.

STEP 2: Technician selects active checkout record (or scans equipment QR).

STEP 3: System fetches checkout details; checks if overdue.

STEP 4: Technician selects condition on return: Good, Damaged, Lost.

STEP 5: If Damaged/Lost: Technician enters damage notes (required).

STEP 6: POST /api/checkouts/:id/return.

STEP 7: Backend updates checkouts status to 'RETURNED'.

STEP 8: If condition = 'Good': Update equipment status to 'Available'.

STEP 9: If condition = 'Damaged': Update equipment status to 'Under Repair'; create alert.

STEP 10: Frontend shows success; prints return receipt.

6.4 Overdue Alert Generation Flow

STEP 1: Trigger event: Daily scheduled job OR application startup.

STEP 2: System queries checkouts: WHERE status = 'ACTIVE' AND due_date < CURRENT_DATE.

STEP 3: For each overdue checkout: Check if alert already exists.

STEP 4: If no active alert: Create alert record in alerts table.

STEP 5: Severity = 'WARNING' (or 'CRITICAL' if >7 days overdue).

STEP 6: Update dashboard alert count.

STEP 7: Frontend displays alert badge; Technician clicks to view list.

STEP 8: Technician contacts borrower or extends due date; marks alert as resolved.

7. NON-FUNCTIONAL REQUIREMENTS

7.1 Performance Requirements

ID	Requirement	Target Metric
NFR-PERF-01	Application startup time	< 3 seconds from launch to login screen
NFR-PERF-02	Page/screen load time	< 1 second for all screens after login
NFR-PERF-03	Database query response time	< 500ms for 95th percentile of queries
NFR-PERF-04	Search results display	< 2 seconds for searches returning up to 1000 items
NFR-PERF-05	Report generation time	< 5 seconds for standard reports
NFR-PERF-06	Memory usage	Application uses < 500MB RAM during normal operation

7.2 Security Requirements

ID	Requirement	Implementation
NFR-SEC-01	Passwords never stored in plaintext	bcrypt hash, salt rounds >= 12
NFR-SEC-02	Protection against SQL injection	TypeORM parameterized queries only

NFR-SEC-03	**Protection against XSS**	**React JSX escaping + Input sanitization**
NFR-SEC-04	**Admin routes protected by RBAC**	**JWT role claim verified server-side on every admin API call**
NFR-SEC-05	**Database file protected**	**Stored in user AppData folder; optional encryption**

7.3 Reliability & Availability Requirements

ID	Requirement	Target
NFR-REL-01	**Application uptime**	**99% availability during lab hours (8 AM - 6 PM)**
NFR-REL-02	**Automated database backups**	**Daily SQLite dump to /backups folder; 30-day retention**
NFR-REL-03	**Data integrity**	**Zero data loss from system crashes; ACID compliance for all transactions**
NFR-REL-04	**Offline Operation**	**Full functionality without internet connectivity; all data stored locally**

7.4 Usability & Accessibility Requirements

ID	Requirement	Notes
NFR-USE-01	**UI is fully responsive**	**Adapts to screen resolutions from 1280x720 to 1920x1080**
NFR-USE-02	**Keyboard navigation**	**All functions accessible via keyboard shortcuts**
NFR-USE-03	**Error prevention**	**Form validation prevents invalid data entry; confirmation dialogs for destructive actions**
NFR-USE-04	**Help and documentation**	**Contextual help available on all screens; user manual accessible offline**

8. DEPLOYMENT & TECHNICAL INFRASTRUCTURE

8.1 Desktop Installer Deployment

The entire application is packaged as a native desktop installer using Electron Builder.

Installation Process:

6. **Download LabInventory-Setup-1.0.0.exe (Windows) or .dmg (Mac).**
7. **Run installer executable.**
8. **Accept license agreement.**
9. **Choose installation location (default: C:\Program Files\Lab Inventory\).**
10. **Click "Install".**
11. **Launch application from Desktop shortcut.**
12. **On first launch, application creates data directory in user's AppData folder.**

8.2 Production Machine Specification

Parameter	Specification
Operating System	**Windows 10 (64-bit) or later; macOS 11 or later**
Processor	**Intel Core i3 or equivalent (dual-core 2.0 GHz)**
RAM	**4 GB minimum; 8 GB recommended**
Storage	**2 GB free space for application; 10 GB for database/backups**
Display	**1280x720 resolution minimum**
Network	**Not required (offline operation)**

8.3 Data Directory Structure

```
LabInventory/
├── lab_inventory.db          # Main SQLite database
├── config.json               # Application settings
├── backups/                  # Automated backup files
├── exports/                  # Exported reports (CSV, Excel, PDF)
├── logs/                     # Application logs
└── uploads/                  # Equipment photos and receipts
```

9. DATABASE SCHEMA OVERVIEW

9.1 Entity Relationship Overview

Entity (Table)	Key Fields	Relationships
users	id, username, email, password_hash, role, department, is_active	Has many: checkouts, activity_logs
categories	id, name, description, parent_id	Has many: equipment
equipment	id, name, serial_number, category_id, model, purchase_date, warranty_expiry, location, condition, status	Belongs to: categories; Has many: checkouts, maintenance
checkouts	id, equipment_id, borrower_id, checkout_date, due_date, return_date, status, condition_on_return	Belongs to: equipment, users (borrower)
maintenance	id, equipment_id, maintenance_type, scheduled_date, completed_date, cost, notes	Belongs to: equipment
alerts	id, type, equipment_id, checkout_id, message, severity, is_read, created_at	Belongs to: equipment, checkouts
activity_logs	id, user_id, action_type, entity_type, entity_id, old_value, new_value, created_at	Belongs to: users

9.2 Key Table Definitions

USERS Table

Column	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY	Unique user identifier
Username	VARCHAR(50)	UNIQUE NOT NULL	Login username
email	VARCHAR(100)	UNIQUE NOT NULL	User email address
password_hash	VARCHAR(255)	NOT NULL	Bcrypt hashed password
role	VARCHAR(20)	NOT NULL	Role: admin, technician, faculty, student
department	VARCHAR(50)	NULL	Department affiliation
is_active	BOOLEAN	NOT NULL DEFAULT true	Account active status

EQUIPMENT Table

Column	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY	Unique equipment identifier
serial_number	VARCHAR(100)	UNIQUE NOT NULL	Unique serial number
name	VARCHAR(200)	NOT NULL	Equipment name
status	VARCHAR(20)	NOT NULL	Status: Available, Checked Out, Under Repair, Retired, Lost
location	VARCHAR(50)	NOT NULL	Physical location (Lab A/B/C)
condition	VARCHAR(20)	NOT NULL DEFAULT 'Good'	Condition: New/Good/Fair/Poo

CHECKOUTS Table

Column	Data Type	Constraints	Description
id	INTEGER	FOREIGN KEY	Unique checkout identifier
equipment_id	INTEGER	FOREIGN KEY	Equipment foreign key
borrower_id	INTEGER	FOREIGN KEY	Borrower foreign key
due_date	DATE	NOT NULL	Expected return date
return_date	DATE	NULL	Actual return date
status	VARCHAR(20)	NOT NULL	Status: ACTIVE, RETURNED, OVERDUE
condition_on_return	VARCHAR(20)	NULL	Condition when returned: Good/Damaged/Lost

10. AGILE DEVELOPMENT PLAN & TIMELINE

10.1 Agile Framework Overview

Ceremony	Frequency	Duration	Participants	Purpose
Sprint Planning	Start of each sprint	2 hours	Entire team + Supervisor	Define sprint goals and commit to backlog items
Daily Standup	Every weekday	15 minutes	Development team	Sync progress, identify blockers, plan day
Sprint Review	End of each sprint	1 hour	Team + Supervisor	Demo completed work, gather feedback
Sprint Retrospective	After review	45 minutes	Development team	Reflect on process, identify improvements

10.2 Sprint Breakdown

Sprint	Duration	Primary Focus	Key Deliverable
Sprint 0		**Foundation & Setup**	**Repo setup, team workflow, initial backlog**
Sprint 1		**Authentication**	**Login/Register, RBAC, JWT auth**
Sprint 2		**Equipment Catalog**	**CRUD equipment, search/filter, categories**
Sprint 3		**Checkout Workflow**	**Checkout/return, status updates, history**
Sprint 4		**Maintenance & Alerts**	**Maintenance tracking, overdue alerts**
Sprint 5		**Reporting**	**Utilization, loss/damage, export reports**

Sprint 6		**Dashboard & Backup**	**Dashboard UI, automated backups**
Sprint 7		**Testing & Release**	**Final testing, documentation, demo**

10.3 Definition of Done (DoD)

A user story or task is considered Done when ALL of the following are true:

✅ Code Quality:

- **Code follows ESLint/Prettier standards; no warnings or errors**
- **No console.log, debugger, or commented-out code in production files**
- **JSDoc comments added for public functions and complex logic**

✅ Testing:

- **Unit tests written and passing for new/modified code**
- **Integration tests added for new API endpoints**
- **Test coverage for modified files >= 80%**

✅ Functionality:

- **All acceptance criteria from the user story met**
- **Feature works correctly with valid and invalid inputs**
- **Error handling implemented (user-friendly messages, no crashes)**

✅ Documentation:

- **API documentation updated if endpoints changed**
- **User manual updated if user-facing behavior changed**

✅ Review & Merge:

- **Code reviewed by at least one peer (Project Lead + one other)**
- **All review comments addressed or documented**
- **CI pipeline passes (lint, test, build)**

- **Merged to develop branch via Pull Request**

11. TEAM ROLES & RESPONSIBILITIES

11.1 Agile Team Structure (6 Members)

Role	Member	Primary Responsibilities	Key Deliverables
Project Lead / Scrum Master	**Princeton**	**Sprint planning; backlog management; stakeholder communication; code review oversight**	**Sprint plans; burndown charts; risk log; final project report**
Frontend Developer	**Regestopher**	**React component development; UI/UX implementation; state management; responsive design**	**Login/Register UI; Equipment forms; Dashboard; Responsive layouts**
Backend Developer	**Philip**	**Nest.js module development; API design; business logic implementation; authentication; testing**	**Auth API; Equipment API; Checkout API; Swagger docs**
Database Developer	**Oliver**	**SQLite schema design; TypeORM entities; migrations; data integrity; query optimization**	**Database schema; Entity classes; Migration scripts; Indexes**
QA/Testing Engineer	**Ernest**	**Test strategy; unit/integration/E2E tests; bug tracking; acceptance criteria validation**	**Test suites; Bug reports; Test coverage reports; UAT scripts**
DevOps/Build Engineer	**Alice**	**Electron packaging; installer creation; CI/CD pipeline; auto-updater; deployment documentation**	**Build scripts; GitHub Actions workflow; Installers; Deployment guide**

11.2 Collaboration Workflow

Daily Standup Format (15 minutes):

- **What did I complete yesterday?**
- **What will I work on today?**
- **What blockers do I have?**

Pull Request Process:

- **Developer creates feature branch from develop.**
- **Implements changes with tests.**
- **Opens PR with description and linked issue.**
- **At least one peer review required (Project Lead + one other).**
- **CI checks must pass (linting, tests).**
- **Merge to develop after approval.**

11.3 Communication Channels

Channel	Purpose	Frequency	Participants
GitHub Issues	**Task tracking, bug reports, feature requests**	**As needed**	**Entire team**
GitHub Projects	**Sprint backlog, task board, progress tracking**	**Daily Updates**	**Entire team**
WhatsApp Group	**Quick questions, daily standup sync, urgent blockers**	**Daily**	**Entire team**
Weekly Sync Call	**Sprint planning, review, retrospective, stakeholder updates**	**Weekly**	**Team + Supervisor**
Email	**Formal documentation, submission, external communication**	**As needed**	**Project Lead + Supervisor**

12. ACCEPTANCE CRITERIA

The project is considered complete when ALL of the following criteria are verified during User Acceptance Testing (UAT).

12.1 Functional Acceptance Criteria

- **A new user can be created by Admin, log in successfully, and access authorized features.**
- **A Lab Technician can add new equipment with unique serial number; duplicate entry is prevented.**
- **A Lab Technician can record equipment checkout with borrower and due date; status updates to "Checked Out".**
- **A Lab Technician can record equipment return with condition check; status updates based on condition.**
- **Overdue checkouts are flagged and alert generated on dashboard.**
- **Maintenance can be scheduled and recorded; history viewable per equipment.**
- **Equipment Utilization Report generates correctly with filters.**
- **Loss/Damage Report lists all incidents in date range.**
- **Reports can be exported to CSV and Excel.**
- **Automated backup created daily; manual backup creates file successfully.**
- **Backup file can be restored; restored database contains all data.**
- **All core pages render correctly and are fully usable on Lab Machine (Windows 10/11).**

12.2 Non-Functional Acceptance Criteria

- **Application starts in < 3 seconds.**
- **Search returns results in < 2 seconds for 1000 items.**
- **Passwords stored as bcrypt hashes (not plaintext).**
- **Unauthorized access to admin routes returns 403.**
- **Application does not crash on invalid input.**
- **Database transactions are atomic (all or nothing).**
- **Backup succeeds 100% of the time.**
- **Application works offline (no internet required).**
- **Installer works on clean machine (no dependencies missing).**

- **Code coverage >= 80% for core modules.**

12.3 Documentation Acceptance Criteria

- **User manual covers all features.**
- **Installation guide clear and complete.**
- **Code documentation (JSDoc) present for public functions.**
- **README file in repository.**
- **Changelog maintained.**

12.4 Code Quality Acceptance Criteria

- **No ESLint errors or warnings.**
- **Code formatted with Prettier.**
- **Unit test coverage >= 80%.**
- **All tests pass in CI pipeline.**
- **No critical security vulnerabilities (npm audit).**
- **Git repository clean (no secrets committed).**
- **Branch protection rules enforced.**

13. APPROVAL & SIGN-OFF

By signing below, each stakeholder confirms they have read this Software Requirements Specification, understand the scope of work described, and authorize development to proceed under the terms herein.

Name	Role	Signature	Date(DD\MM\YY)
	Academic Supervisor / Professor		
	Lab Technician Representative		
	Project Lead / Student Developer		
	Frontend Developer		
	Backend Developer		
	Database Developer		
	QA/Testing Engineer		
	DevOps/Build Engineer		

Document Version: 1.0

Last Updated: March 2026

Next Review: End of Sprint 3 (mid-project review)

SDLC Methodology: Agile (Scrum) with 2-week sprints